

BANKING DATA ANALYSIS AND BUSINESS INTELLIGENCE DASHBOARD

SQL Server and Microsoft Power BI

PROJECT DOCUMENTATION

Prepared by: Dhanushya R

Data Analytics Project

2026

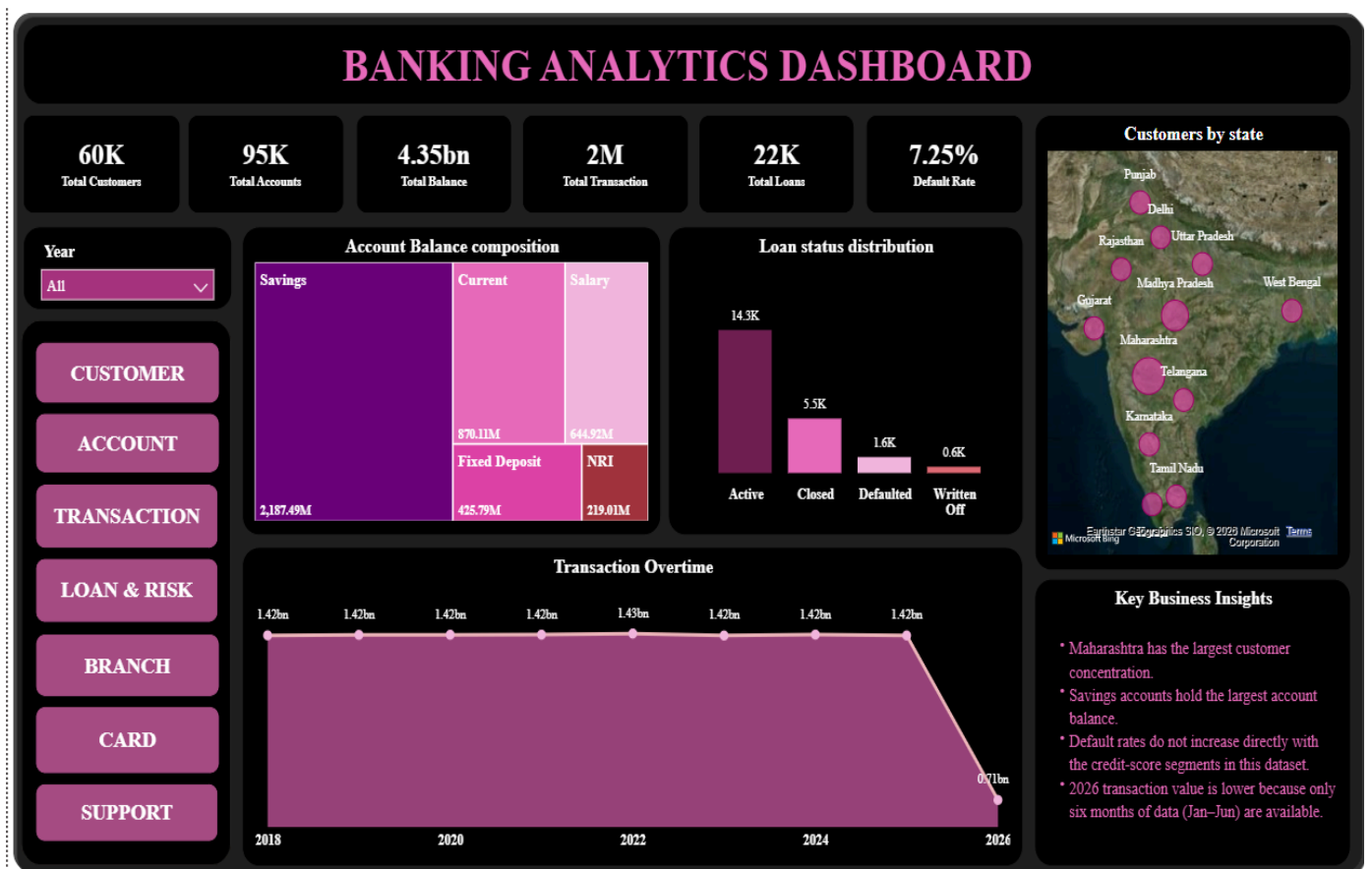


Figure 1.1

TABLE OF CONTENTS

S. No.	Chapter / Section	Page No.
1	Abstract	4
2	Introduction	5
3	Background and Business Context	6
4	Problem Statement	7
5	Aim and Objectives	8
6	Scope of the Project	9
7	Tools and Technologies	10
8	Dataset Description	11
9	Database Structure and Relationships	12
10	Data Quality and Validation	13
11	Project Methodology	13
12	Business Problem Areas	16
13	SQL Analysis	17
14	Power BI Data Model	19
15	DAX Measures and Calculations	21

16	Dashboard Design and Navigation	23
17	Executive Dashboard	24
18	Customer Analysis	24
19	Account Analysis	25
20	Transaction Analysis	25
21	Loan and Risk Analysis	26
22	Branch Analysis	26
23	Card Analysis	27
24	Support Analysis	27
25	Key Business Insights	28
26	Recommendations	29
27	Project Outcomes	29
28	Limitations	30
29	Future Enhancements	30
30	Conclusion	31
31	References	31
32	Appendix A – SQL Queries	32
33	Appendix B – DAX Measures and Calculations	54

1. ABSTRACT

This project focuses on analysing a large-scale banking dataset using SQL Server and Microsoft Power BI to generate meaningful business insights and an interactive dashboard. The dataset contains 5.2M+ records, including approximately 60K customers, 95K accounts, 22K loans, 65K cards, 2M account transactions, 3M card transactions and 25K support tickets.

The project covers Customer, Account, Transaction, Loan & Risk, Branch, Card and Support Analysis, along with an Executive Dashboard. SQL is used for data exploration, joins, aggregation, subqueries, CTEs and window functions, while Power BI and DAX are used for data modeling, KPI creation and interactive visualization.

The final dashboard enables users to analyse banking performance, customer behaviour, financial activity, loan risk, branch performance, card usage, fraud and customer support. The project demonstrates an end-to-end data analytics and Business Intelligence workflow, from raw data and SQL analysis to visualization, insights and recommendations.

2. INTRODUCTION

The banking industry generates large amounts of data through customer accounts, transactions, loans, cards, branches and support services. Analysing this data is important for understanding customer behaviour, monitoring financial performance and identifying potential risks.

This project uses a 5.2M+ record relational banking dataset containing multiple interconnected tables. SQL Server is used to explore and analyse the data, while Power BI is used to build an interactive Business Intelligence dashboard.

The analysis focuses on seven major areas: Customer, Account, Transaction, Loan & Risk, Branch, Card and Support Analysis. Relevant KPIs and visualizations are created for each area to convert raw banking data into clear and actionable business insights.

The project follows the workflow:

Data Understanding → Business Problems → KPIs → SQL Analysis → Validation → Power BI → DAX → Dashboard → Insights → Recommendations

3. BACKGROUND AND BUSINESS CONTEXT

The banking industry generates a large volume of data through customer accounts, financial transactions, loans, credit cards, branches, repayments and customer support activities. This data contains valuable information about customer behaviour, financial performance, loan risk and operational efficiency. However, analysing large and interconnected datasets manually can make it difficult to identify important patterns and business trends.

With the growth of digital banking, financial institutions need effective data analytics and Business Intelligence solutions to convert raw data into meaningful information. SQL is useful for querying, filtering, aggregating and analysing relational data, while Power BI helps transform the analysed data into interactive dashboards and visual reports.

This project uses a large-scale relational banking dataset containing 5.2M+ records across multiple banking domains, including customers, accounts, transactions, loans, cards, branches and support tickets. The project combines SQL Server, SQL analysis, data modelling, DAX and Power BI visualization to analyse the banking data from different business perspectives.

The analysis is divided into Customer, Account, Transaction, Loan & Risk, Branch, Card and Support Analysis, along with an Executive Dashboard. The project aims to provide a clear view of banking operations and help identify important patterns such as customer concentration, account balances, transaction activity, loan defaults, branch performance, card spending, fraud and support performance.

4. PROBLEM STATEMENT

Large banking datasets contain information from multiple interconnected areas, making it challenging to obtain useful business insights directly from raw data. Without proper analysis, it can be difficult to understand customer behaviour, monitor account and transaction activity, evaluate loan risk, compare branch performance and identify unusual card transactions.

The absence of a centralized analytical dashboard can also make it difficult for decision-makers to quickly monitor important banking KPIs and compare performance across different business areas.

Therefore, there is a need for a structured analytical solution that can:

- Analyse large volumes of banking data efficiently.
- Identify important customer and account patterns.
- Monitor transaction activity and financial values.
- Analyse loan performance and default risk.
- Compare branch-level performance.
- Understand card usage and fraudulent transactions.
- Evaluate customer support activity and service performance.
- Present important KPIs and insights through an interactive dashboard.

This project addresses these challenges by using SQL Server for data analysis and Power BI for visualization and Business Intelligence reporting.

5. AIM AND OBJECTIVES

5.1 Aim of the Project

The main aim of this project is to analyse a large-scale banking dataset using SQL Server and Power BI to identify meaningful business insights and develop an interactive dashboard for effective data-driven decision-making.

5.2 Objectives

The key objectives of the project are:

1. To explore and understand the structure of the banking database and its relationships.
2. To analyse customer demographics, income and credit-score information.
3. To evaluate account types, balances and account status.
4. To analyse transaction volume, transaction value and activity patterns.
5. To identify loan performance and credit-risk patterns, including default rates.
6. To compare branch performance based on customers, balances, transactions and loan portfolios.
7. To analyse card usage, spending categories and fraudulent transactions.
8. To evaluate customer support activity, resolution time and satisfaction.
9. To use SQL queries and DAX measures for meaningful data analysis.
10. To develop an interactive Power BI dashboard with KPIs, visualizations and slicers.
11. To derive actionable business insights and recommendations from the analysis.

6. SCOPE OF THE PROJECT

The scope of this project covers the analysis and visualization of banking data across multiple business areas. The project uses a relational banking dataset containing **5.2M+ records** and focuses on converting raw data into useful business information.

The project includes:

- Customer Analysis – demographics, income, credit score and customer distribution.
- Account Analysis – account types, balances, status and account activity.
- Transaction Analysis – transaction volume, value, channels and trends.
- Loan & Risk Analysis – loan amounts, loan types, default rates and credit-risk segments.
- Branch Analysis – customer concentration, balances, transactions and loan portfolios.
- Card Analysis – card types, spending patterns and fraud analysis.
- Support Analysis – support tickets, issue types, resolution time and satisfaction.
- Executive Analysis – key KPIs and consolidated business insights.

7. TOOLS AND TECHNOLOGIES

The following tools and technologies were used to develop the Banking Data Analysis and Business Intelligence Dashboard

Tool / Technology	Purpose
SQL Server	Database management, storage and execution of SQL queries
SQL	Data extraction, filtering, sorting, aggregation and analysis
Power BI	Interactive dashboard development and business data visualization
DAX	Creating calculated measures, calculated columns and KPIs
Microsoft Excel	Initial data handling, inspection and preparation
CSV	Storing and importing structured banking data
Power BI Data Model	Creating relationships between different banking tables
Date Table	Supporting time-based analysis and year-wise filtering
Power BI Slicers	Providing interactive filtering across dashboard visuals
Power BI KPI Cards	Displaying important banking performance indicators
Power BI Charts	Visualizing trends, comparisons, distributions and patterns
GitHub	Project version control, documentation and portfolio presentation

8. DATASET DESCRIPTION

The dataset contains approximately 60K customers, 95K accounts, 22K loans, 65K cards and millions of transaction/card-transaction records across multiple years.

8.1 Major Tables

Table Name	Description	Key fields
customers	Customer master data	customer_id, name, gender, city, state, annual_income, join_date, credit_score
accounts	Bank accounts details	account_id, customer_id, branch_id, account_type, balance, open_date, status
branches	Bank branch details	branch_id, branch_name, city, state, opened_date
transactions	Account transactions details	transaction_id, account_id, txn_date, txn_type, channel, amount
loans	Loan information	loan_id, customer_id, loan_type, loan_amount, status, start_date
loan_payments	Loan repayments details	payment_id, loan_id, payment_date, amount_paid, late_payment_flag
cards	Card master data	card_id, customer_id, account_id, card_type, credit_limit, issue_date, expiry_date, status
card_transactions	Card transactions details	card_txn_id, card_id, txn_date, merchant_category, amount, is_fraud
support_tickets	Customer support details	ticket_id, customer_id, issue_type, status, date_opened, date_resolved, satisfaction_score
employees	Employees details	employee_id, name, branch_id, role, hire_date, salary

9. DATABASE STRUCTURE AND RELATIONSHIPS

The banking database follows a relational structure in which multiple tables are connected through primary keys and foreign keys. The relationships were created and validated in SQL Server to ensure proper connectivity between the different banking entities.

- Primary keys and foreign keys were created and validated.
- One-to-many relationships were established between related tables.
- The database structure was reviewed to ensure proper table connectivity.
- The relationships support efficient querying and analysis of customers, accounts, transactions, loans, cards and support activities.

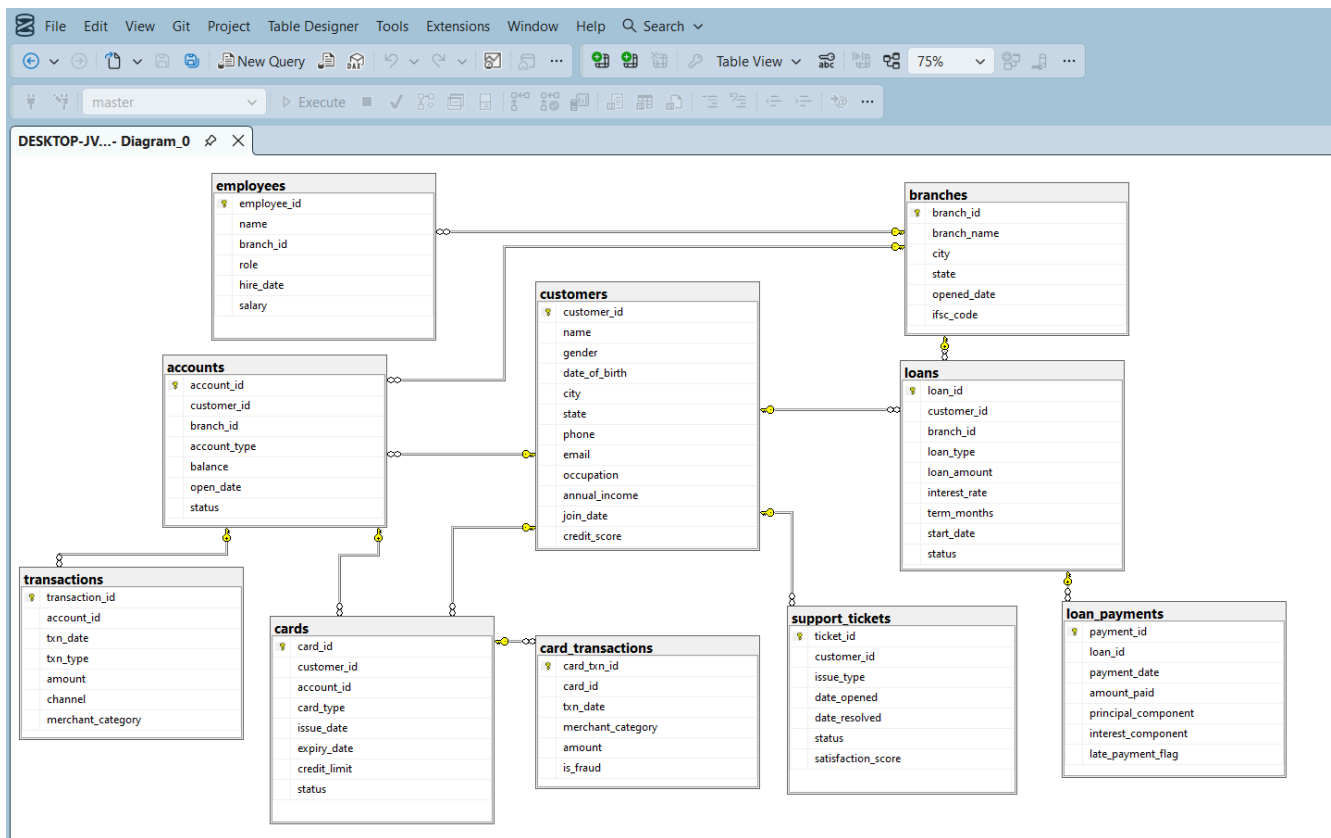


Figure 1.2

10. DATA QUALITY AND VALIDATION

The dataset was validated before dashboard development to ensure data reliability and consistency. Key analytical fields, data values and relationships were reviewed, and the dataset was considered suitable for the final analysis and dashboard.

- Duplicate checks were performed.
- Null-value checks were performed.
- Primary and foreign-key relationships were reviewed.
- SQL results were compared with corresponding Power BI measures.
- Date ranges were checked for consistency.
- Relationship ambiguity was investigated and resolved.

Important Temporal Note: The transaction data ends on 30 June 2026. Therefore, 2026 contains data only from January to June and should not be interpreted as a complete-year decline when comparing it with previous years.

11. PROJECT METHODOLOGY

The project follows a structured data analytics and Business Intelligence methodology, beginning with understanding the banking database and ending with dashboard development and business insights.

Methodology Steps

1. Data Understanding

The banking dataset and its tables, fields and relationships were examined to understand the available information.

2. Business Problem Identification

Key business areas such as customers, accounts, transactions, loans, branches, cards and support were identified for analysis.

3. KPI Identification

Relevant KPIs were defined for each business area, such as total customers, account balance, transaction value, loan amount and default rate.

4. SQL Data Analysis

SQL Server was used to perform data extraction, filtering, aggregation, joins, subqueries, CTEs and window-function analysis.

5. Data Validation

SQL results, relationships, date ranges, duplicate records and null values were checked to ensure reliable analysis.

6. Power BI Data Modelling

The validated tables were imported into Power BI and appropriate relationships and the Date Table were established.

7. DAX Development

DAX measures and calculated columns were created for KPIs, percentages, counts and analytical calculations.

8. Dashboard Development

Interactive Power BI pages were developed for Executive, Customer, Account, Transaction, Loan & Risk, Branch, Card and Support Analysis.

9. Business Insights

Dashboard results were analysed to identify important patterns, trends, risks and performance differences.

10. Recommendations

The identified insights were converted into practical recommendations to support better banking decisions.

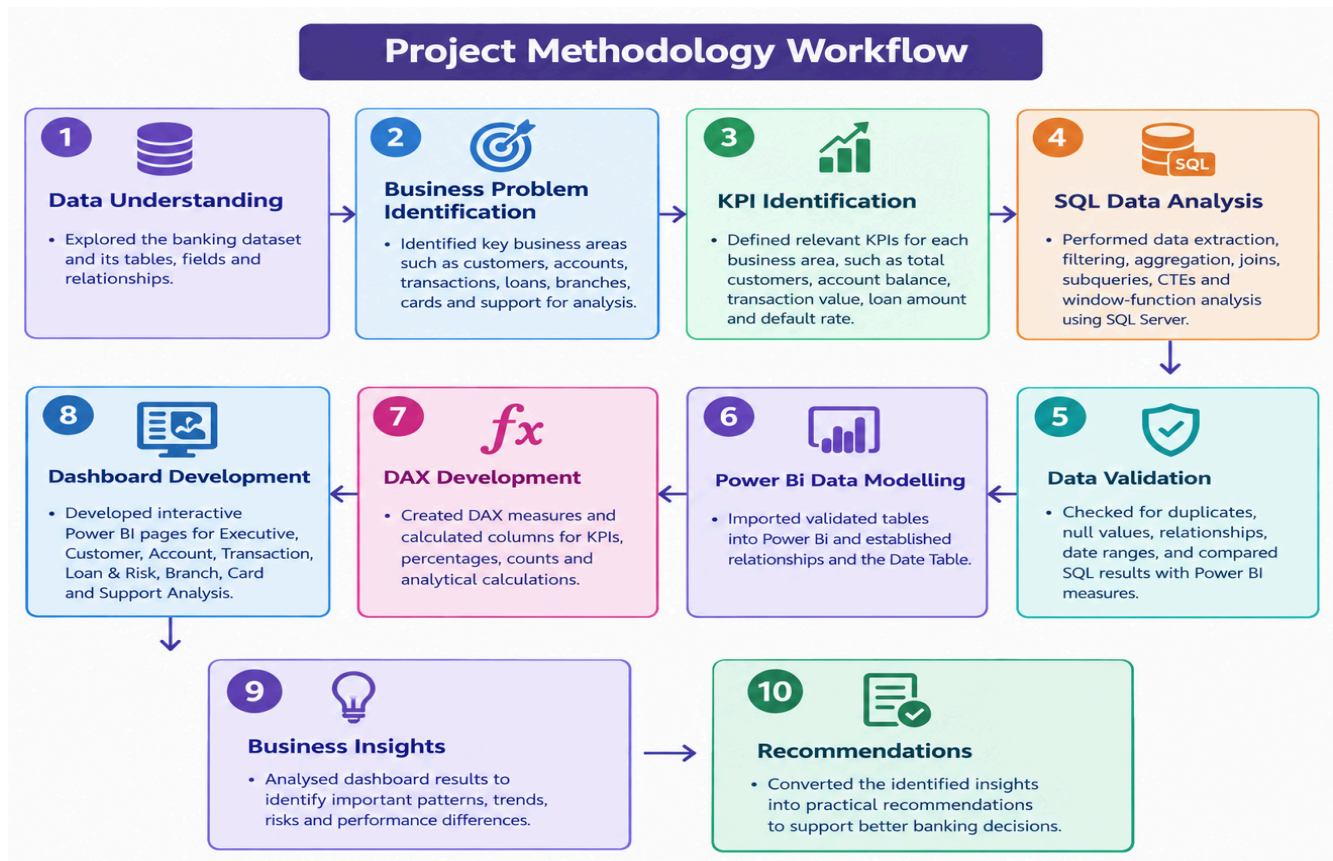


Diagram 1.1

12. BUSINESS PROBLEM AREAS

No.	Business Problem Area	Focus of Analysis
1	Customer Analysis	Customer distribution, demographics, income, credit score and customer growth
2	Account Analysis	Account types, balances, account status and account opening trends
3	Transaction Analysis	Transaction volume, transaction value, channels and transaction trends
4	Loan & Risk Analysis	Loan portfolio, loan types, default rate and credit-risk segments
5	Branch Analysis	Branch-wise customers, balances, transactions and loan portfolios
6	Card Analysis	Card types, card status, spending categories and fraudulent transactions
7	Support Analysis	Support tickets, issue types, resolution time and customer satisfaction
8	Executive Analysis	Overall KPIs, key business insights, trends and performance summary

13. SQL ANALYSIS

SQL Server was used to perform detailed analysis of the banking database. The analysis was carried out across the major business areas identified in the project. SQL queries were used to retrieve, filter, aggregate and combine data from multiple related tables to calculate KPIs and identify important business patterns. The analysis covered the following areas:

13.1 Customer Analysis

Customer data was analysed to understand customer distribution, income levels, credit scores, geographical concentration and customer growth over time.

13.2 Account Analysis

Account data was analysed to examine the distribution of account types, account status, total balances, average balances and account opening trends.

13.3 Transaction Analysis

Transaction records were analysed to evaluate transaction volume, transaction value, transaction types, transaction channels and transaction trends over time.

13.4 Loan and Risk Analysis

Loan data was analysed to identify loan distribution, total loan amounts, average loan amounts, loan status, default rates and credit-risk patterns.

13.5 Branch Analysis

Branch-level analysis was performed to compare the number of customers, accounts, balances, transactions and loan portfolios across different branches.

13.6 Card Analysis

Card data and card transactions were analysed to examine card types, card status, transaction value, spending categories, customer activity and fraudulent transactions.

13.7 Support Analysis

Support ticket data was analysed to identify ticket volume, common issue types, ticket status, resolution time, customer satisfaction and support trends.

SQL Techniques Used

The following SQL techniques were applied during the analysis:

- SELECT statements for retrieving data.
- WHERE clauses for filtering records.
- GROUP BY and ORDER BY for grouping and sorting results.
- Aggregate functions such as COUNT(), SUM(), AVG(), MIN() and MAX() for KPI calculations.
- INNER JOINS for combining data from related tables.
- Subqueries for performing nested and comparative analysis.
- Common Table Expressions (CTEs) for structuring complex queries.
- Window functions such as ROW_NUMBER(), RANK(), LAG() and LEAD() for ranking and trend analysis.
- Date functions such as YEAR(), MONTH() and DATEDIFF() for time-based analysis.

14. POWER BI DATA MODEL

Power BI was used to create a structured data model by importing the validated banking tables from SQL Server. The tables were connected using appropriate relationships so that filters and calculations could flow correctly across the different areas of the banking dataset.

The data model includes the major tables such as Customers, Accounts, Branches, Transactions, Loans, Loan Payments, Cards, Card Transactions, Support Tickets and Employees.

Data Modelling Activities

- Imported the validated SQL Server tables into Power BI.
- Established relationships using primary and foreign keys.
- Defined appropriate one-to-many relationships between related tables.
- Created and connected a common Date Table for time-based analysis.
- Reviewed relationship paths to avoid ambiguous filtering.
- Used appropriate active and inactive relationships where required.
- Validated the model by checking whether slicers, measures and visuals responded correctly.

The Power BI data model provided the foundation for creating DAX measures, KPIs, slicers and interactive dashboard visualizations.

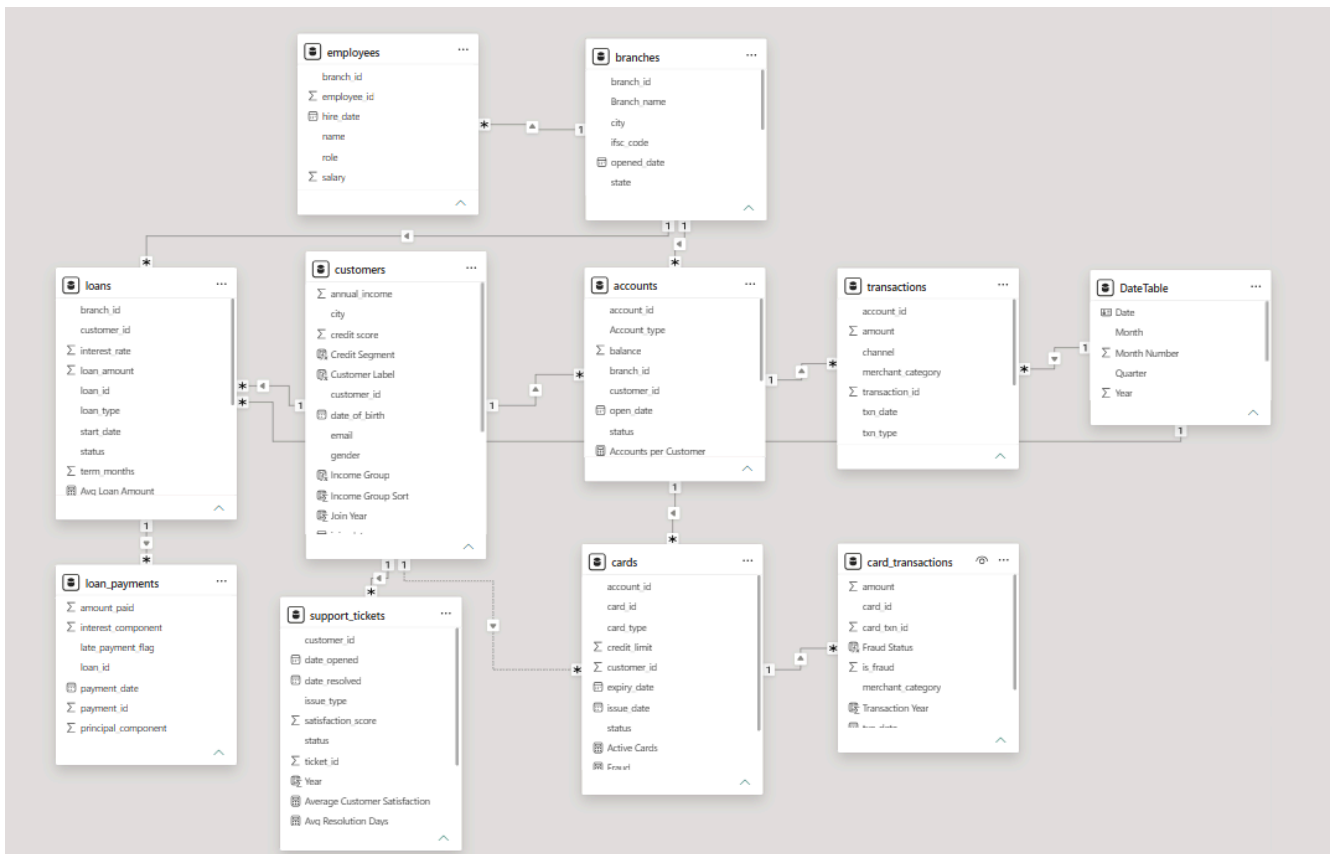


Figure 1.3

15. DAX MEASURES AND CALCULATIONS

DAX (Data Analysis Expressions) was used in Power BI to create measures and calculated columns required for KPI calculations, data analysis and dashboard visualizations. These calculations helped convert the raw banking data into meaningful business metrics.

Measure	Purpose
Total Customers	Calculates the total number of unique customers
Total Accounts	Calculates the total number of bank accounts
Total Account Balance	Calculates the total balance held across accounts
Average Account Balance	Calculates the average account balance
Active Accounts	Calculates the number of active accounts
Total Transactions	Calculates the total number of account transactions
Average Transaction Amount	Calculates the average value of transactions
Total Transaction Value	Calculates the total value of account transactions
Total Loans	Calculates the total number of loans
Total Loan Amount	Calculates the total loan portfolio value
Average Loan Amount	Calculates the average loan amount
Default Rate	Calculates the percentage of loans in default

Total Branches	Calculates the number of unique branches
Total Cards	Calculates the total number of cards
Active Cards	Calculates the number of active cards
Total Card Transaction Value	Calculates the total value of card transactions
Fraudulent Transactions	Calculates the number of fraudulent card transactions
Fraud %	Calculates the percentage of fraudulent transactions
Total Support Tickets	Calculates the total number of support tickets
Average Resolution Days	Calculates the average time taken to resolve tickets
Average Satisfaction Score	Calculates the average customer satisfaction rating

Calculated Columns

Calculated columns were also created where row-level classification was required, such as:

- Credit Segment – categorizes customers into Poor, Good, Fair and Excellent based on credit score.
- Fraud Status – classifies card transactions as Fraud or Legitimate.
- Transaction Year – extracts the year from card transaction dates.
- Support Year – extracts the year from support ticket opening dates

16. DASHBOARD DESIGN AND NAVIGATION

The dashboard uses a dark theme with page-specific accent colours. KPI cards are placed prominently, followed by analytical visuals. Navigation buttons allow movement between business areas.

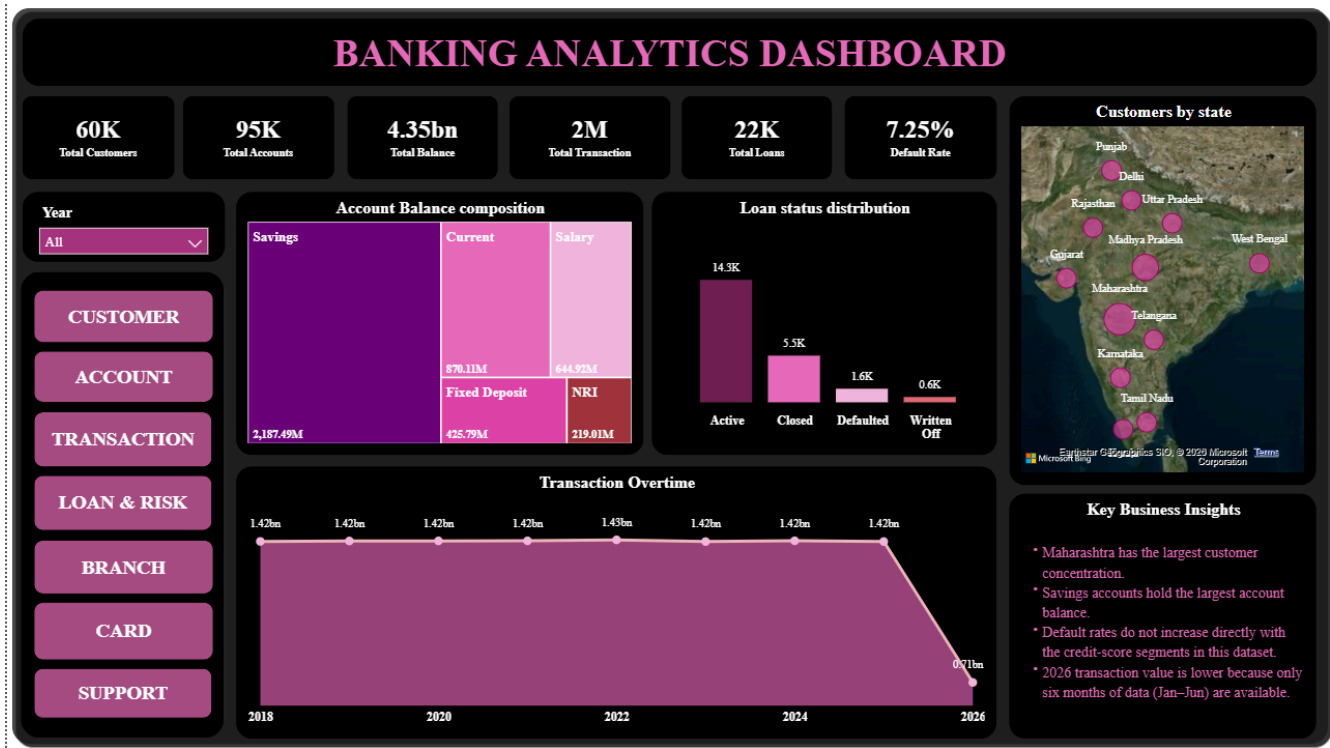


Figure 1.4

17. EXECUTIVE DASHBOARD

Management-level summary of major banking KPIs and key business insights.

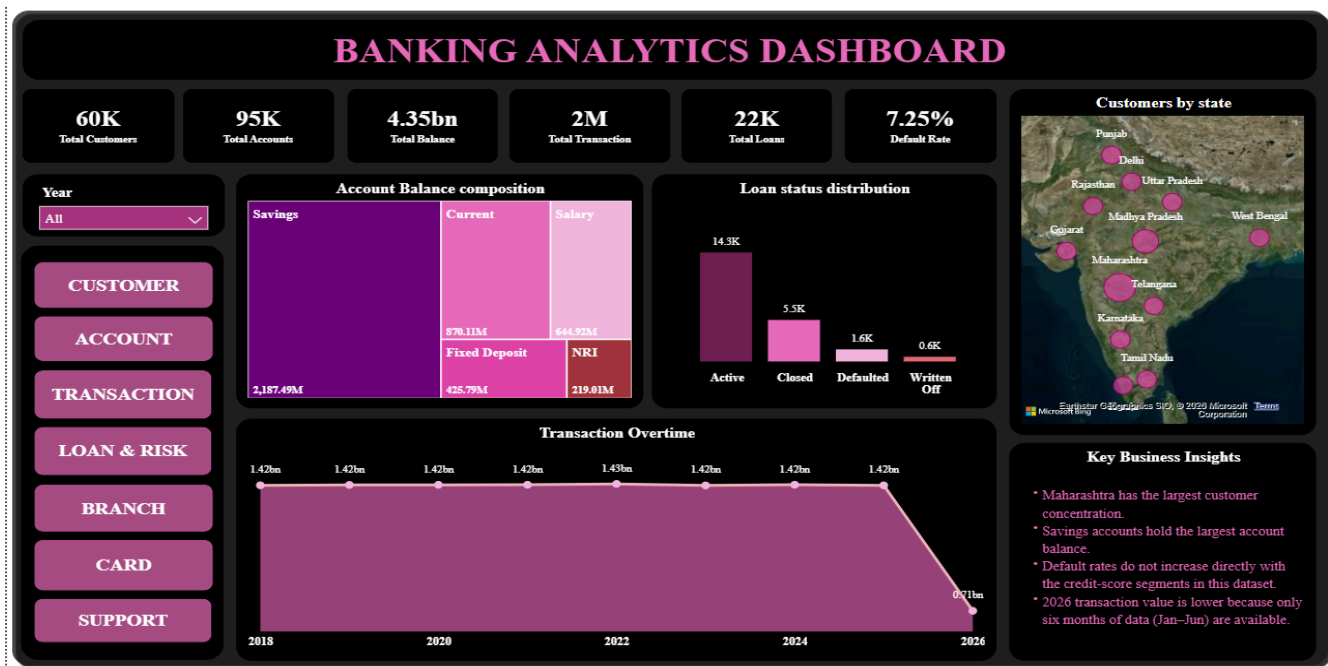


Figure 1.5

18. CUSTOMER ANALYSIS

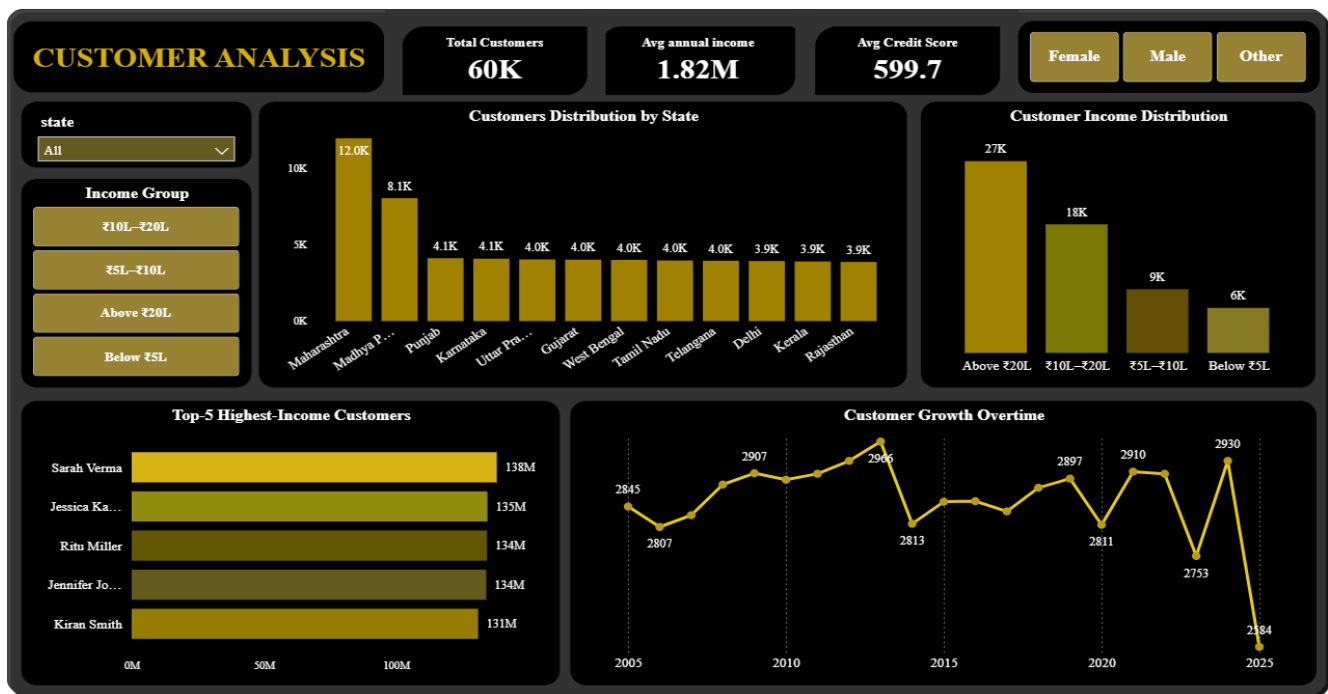


Figure 1.6

19. ACCOUNT ANALYSIS



Figure 1.7

20. TRANSACTION ANALYSIS

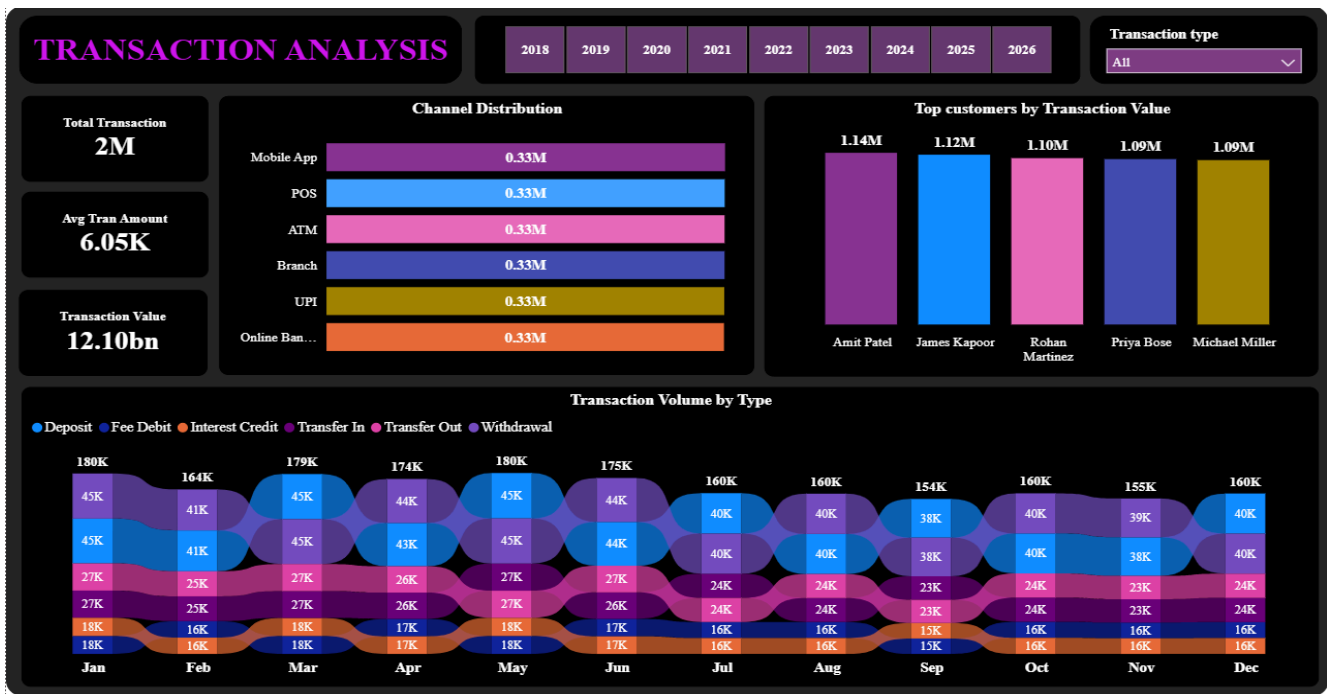


Figure 1.8

21. LOAN AND RISK ANALYSIS

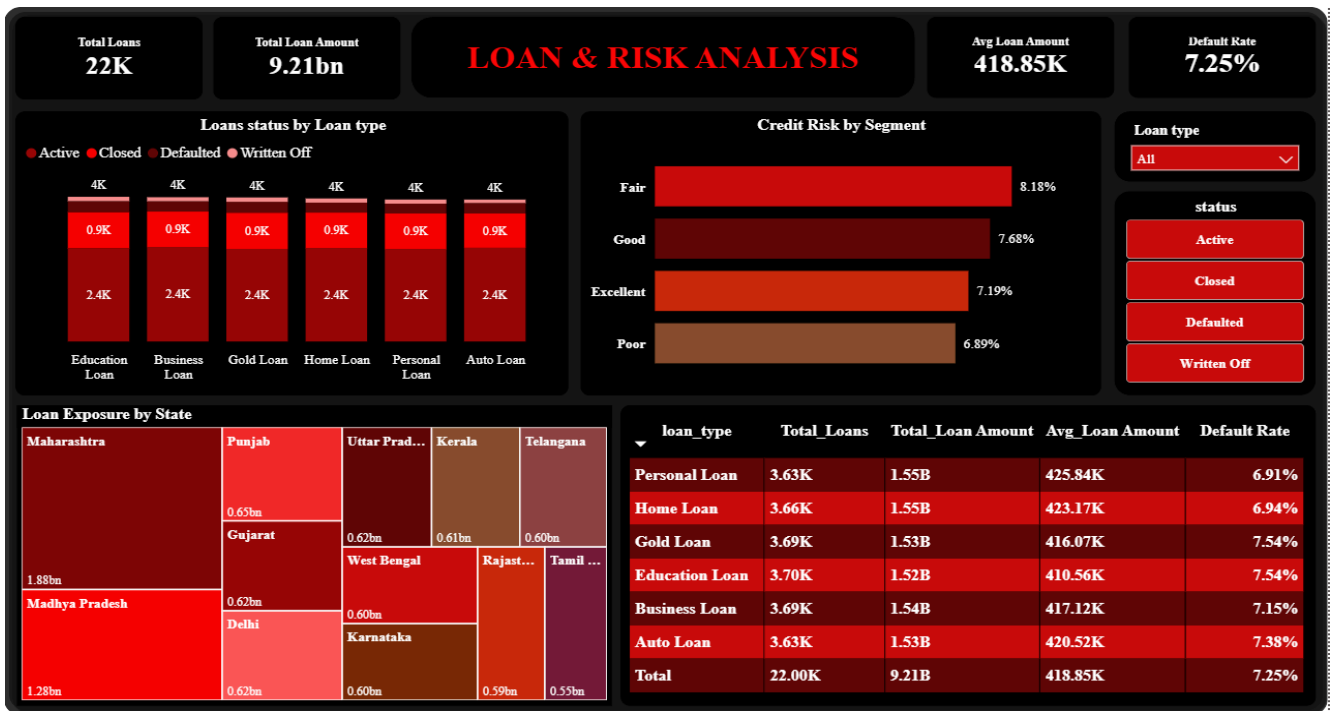


Figure 1.9

22. BRANCH ANALYSIS

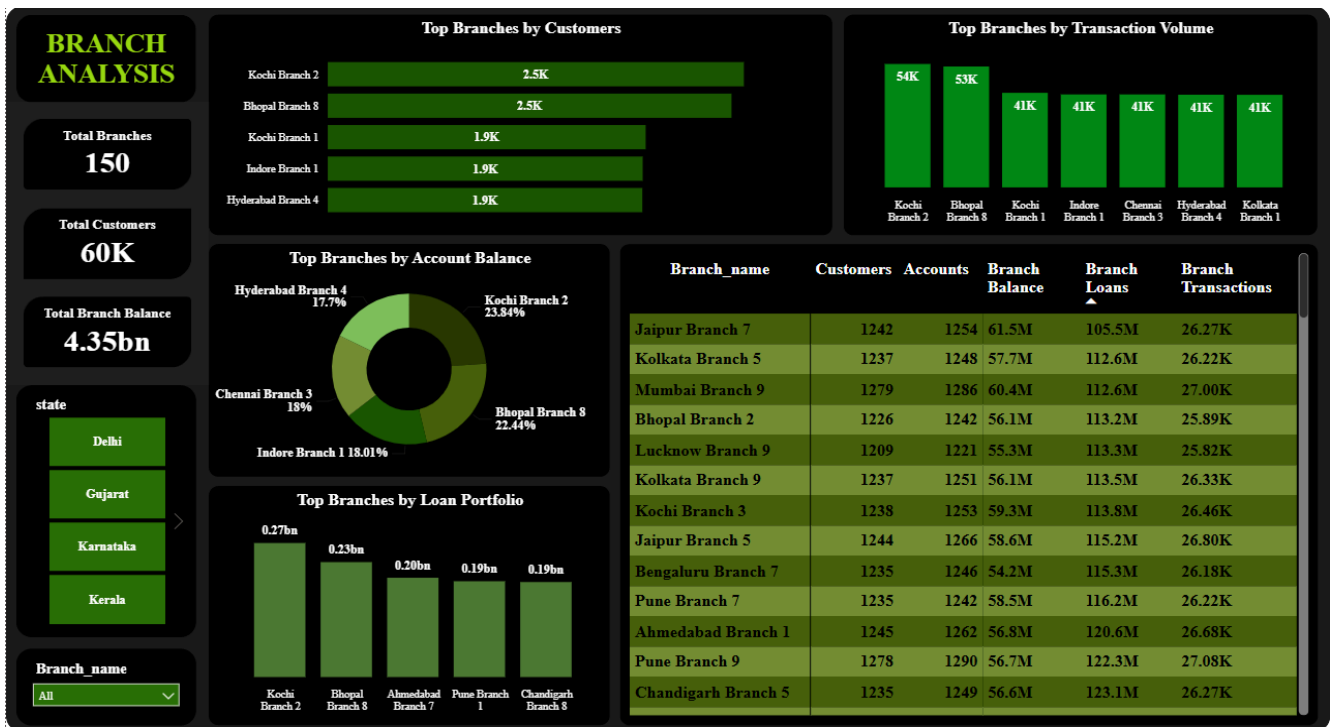


Figure 1.10

23. CARD ANALYSIS

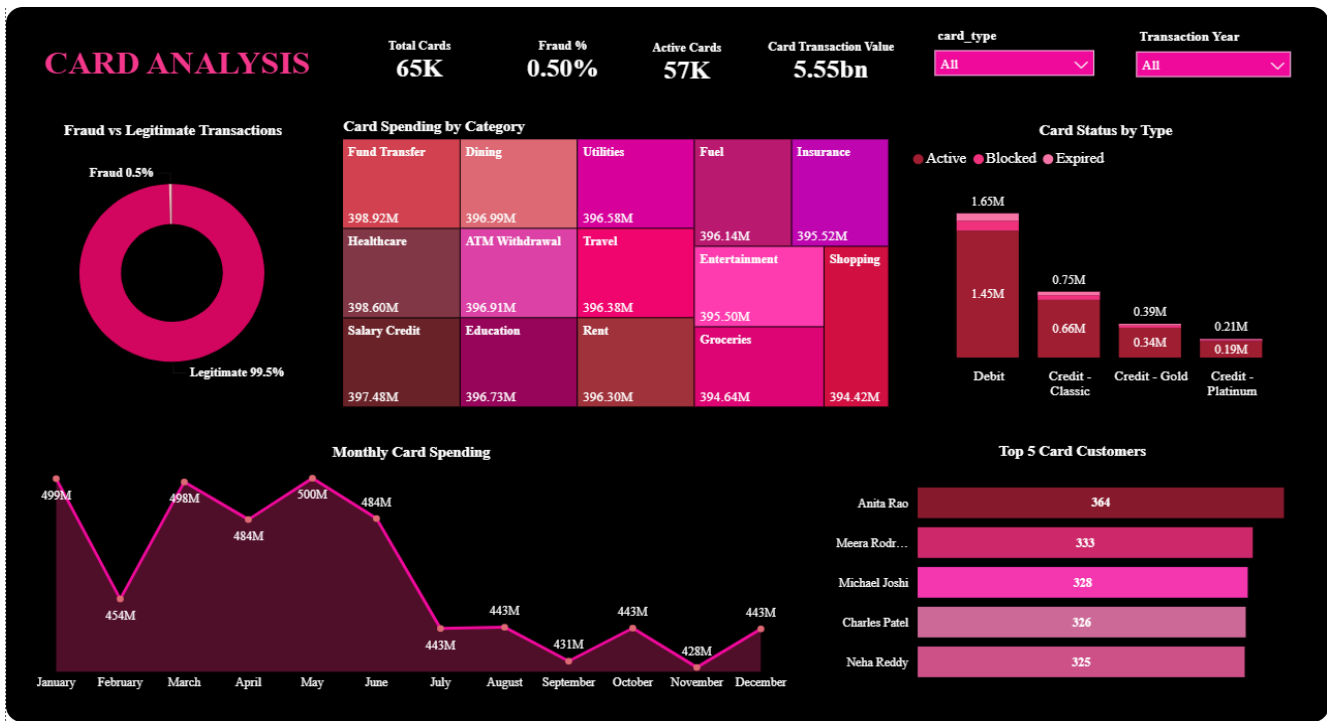


Figure 1.11

24. SUPPORT ANALYSIS

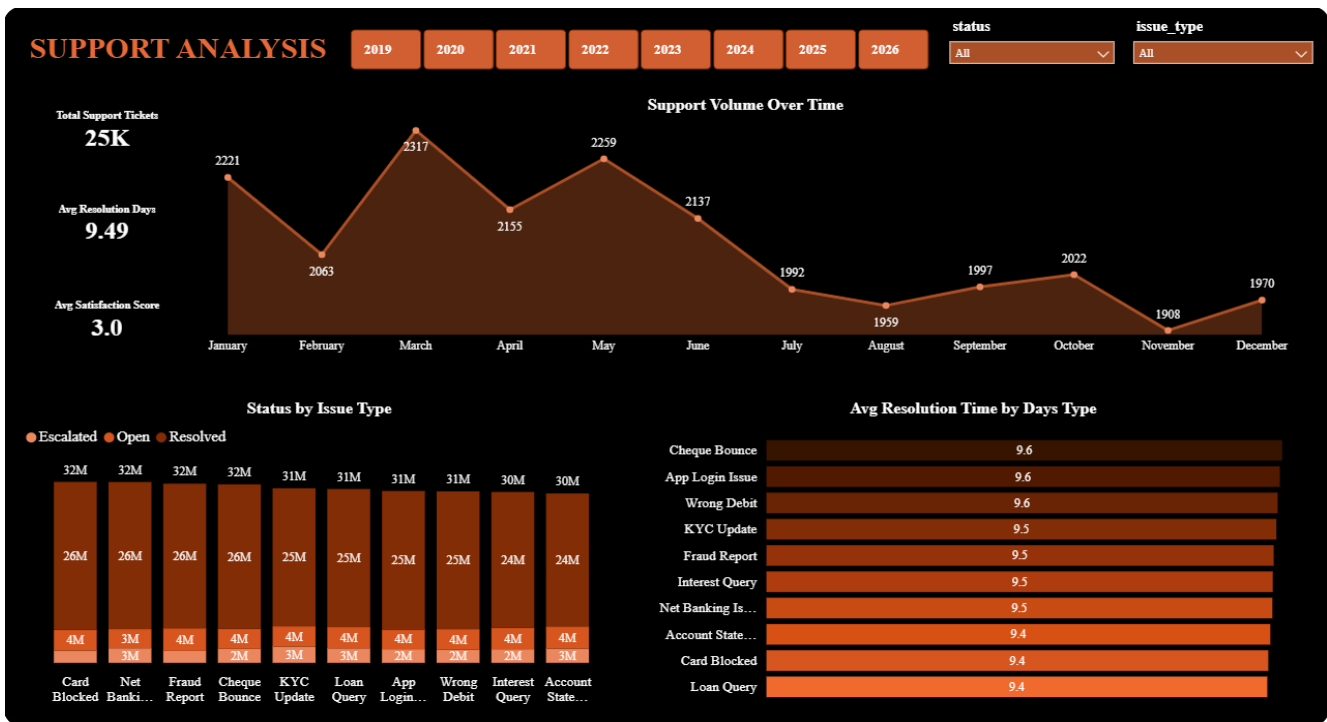


Figure 1.12

25. KEY BUSINESS INSIGHTS

- Maharashtra has the highest customer concentration in the Customer Analysis view.
- The customer base is approximately 60K and the average credit score is approximately 599.7.
- Savings is the largest account category by account count and contributes the largest share of account balance.
- Total account balance is approximately 4.35bn, with active accounts forming the majority.
- Transaction value is approximately 12.10bn and activity is distributed across multiple channels.
- 2026 transaction value appears lower because only January–June 2026 data is available; it is not a valid full-year decline comparison.
- Loan portfolio is approximately 9.21bn with an overall displayed default rate of about 7.25%.
- Credit-segment default rates are relatively close, indicating that credit segment alone does not explain all default behaviour.
- Branch performance varies by customers, balances, transactions and loans; one metric alone should not determine branch performance.
- Card portfolio is approximately 65K cards with approximately 57K active cards.
- Card transaction value is approximately 5.55bn and fraud is about 0.50% of card transactions.
- Card spending is spread across many merchant categories, supporting the use of a treemap.
- Support activity varies by month, with average resolution around 9.49 days and average satisfaction around 3.0 out of 5.

26. RECOMMENDATIONS

- Use customer concentration to support branch acquisition and service planning.
- Monitor savings-account balance concentration because it represents a large share of portfolio value.
- Review transaction-channel behaviour to understand customer preferences and operational demand.
- Use multiple risk factors for loan monitoring instead of relying only on credit-score segment.
- Compare branches across customers, balances, loans and transactions rather than using a single KPI.
- Continue fraud monitoring because a low percentage can still represent many transactions at high volumes.
- Investigate support issue types with higher resolution times.

27. PROJECT OUTCOMES

The project successfully developed an end-to-end banking data analytics and Business Intelligence solution using SQL Server and Power BI.

The major outcomes include:

- Analysed a 5.2M+ record banking dataset.
- Structured and validated a relational banking database.
- Performed detailed SQL analysis across major banking functions.
- Developed DAX measures and calculated columns for business KPIs.
- Created an integrated Power BI data model.
- Developed 8 analytical dashboard areas, including the Executive Dashboard.
- Created interactive visualizations, KPI cards and slicers.
- Identified customer, account, transaction, loan, branch, card and support-related insights.
- Converted analytical findings into actionable recommendations.
- Demonstrated an end-to-end workflow from raw banking data → SQL analysis → Power BI dashboard → business insights.

28. LIMITATIONS

- The dataset is historical and may not represent current real-world banking conditions.
- Metric interpretation depends on the business date selected.
- 2026 transaction data is incomplete through 30 June.
- Observed patterns do not automatically prove causal relationships.
- Customer satisfaction is available only where a score is recorded.

29. FUTURE ENHANCEMENTS

- Automated database refresh.
- Power BI Service publication and scheduled refresh.
- Customer and branch drill-through pages.
- Customer segmentation using clustering.
- Loan default and churn prediction.
- Transaction anomaly detection.
- Year-to-date versus previous-year comparison measures.
- Role-specific date dimensions for multi-date analysis.
- Row-level security for branch/region users.

30. CONCLUSION

This project successfully demonstrates an end-to-end banking data analytics and Business Intelligence workflow using SQL Server and Power BI. The project progresses from relational data understanding and business-problem identification through SQL analysis, data validation, data modelling, DAX calculations, dashboard development, business insights and recommendations.

The final dashboard is organized into Customer, Account, Transaction, Loan & Risk, Branch, Card and Support Analysis, with an Executive Dashboard providing a consolidated management-level summary. Visualizations were selected based on the specific business questions rather than simply repeating chart types, ensuring that each visual provides meaningful analytical value.

The project successfully analysed a 5.2M+ record banking dataset and transformed the data into meaningful KPIs, visualizations and business insights. It demonstrates practical capabilities in SQL, data modelling, DAX, Power BI, data visualization and business interpretation.

The project also highlights the importance of considering analytical context when interpreting results. For example, the 2026 transaction value represents only a six-month period, as the available transaction data ends on 30 June 2026; therefore, it should not be interpreted as a complete-year decline.

Overall, the solution demonstrates how large-scale banking data can be transformed into a structured, interactive and business-focused analytical solution that supports data-driven decision-making.

31. REFERENCES

- Microsoft. *SQL Server Documentation*. [Microsoft SQL Server Documentation](#)
- Microsoft. *Transact-SQL (T-SQL) Documentation*. Transact-SQL Documentation
- Microsoft. *Power BI Documentation*. [Power BI Documentation](#)
- Microsoft. *DAX Function Reference*. [DAX Function Reference](#)
- Microsoft. *Power BI Data Modeling Documentation*. [Power BI Data Modeling](#)
- Microsoft. *Power Query Documentation*. [Power Query Documentation](#)

APPENDIX A – SQL QUERIES

A.1 Database Setup and Data Understanding

```
use BankingDB
```

```
go
```

```
select count(*) as customer_count
```

```
from customers;
```

```
select count(*) as account_count
```

```
from accounts;
```

```
select count(*) as cards_count
```

```
from cards;
```

```
select count(*) as card_transaction_count
```

```
from card_transactions;
```

```
select count(*) as employee_count
```

```
from employees;
```

```
select count(*) as loan_count
```

```
from loans;
```

```
select count(*) as loan_payment_count
```

```
from loan_payments;
```

```
select count(*) as transaction_count
```

```
from transactions;
```

```
select count(*) as branch_count
```

```
from branches;
```

```
select count(*) as support_tickets_count
```

```
from support_tickets;
```

```
--understand the data
```

```
select distinct account_type
```

```
from accounts;
```

```
select distinct status
```

```
from accounts;
```

```
select distinct txn_type
```



```

from transactions;

select distinct channel
from transactions;

select distinct loan_type
from loans;

select distinct status
from loans;

--Understand the date range

select min(txn_date) as first_transaction,
       max(txn_date) as last_transaction
from transactions;

select min(start_date) as first_loan,
       max(start_date) as last_loan
from loans;

select min(payment_date) as first_payment,
       max(payment_date) as last_payment
from loan_payments;

select min(open_date) as first_account,
       max(open_date) as last_account
from accounts;

select min(join_date) as first_customer,
       max(join_date) as last_customer
from customers;

select min(issue_date) as first_issue_card,
       max(issue_date) as last_issue_card
from cards;

```

--frequently value occurs

```
select account_type, count(*) as account_count
from accounts
group by account_type
order by account_count desc;
```

```
select status, count(*) AS account_count
from accounts
group by status
order by account_count desc;
```

```
select txn_type, count(*) AS transaction_count
from transactions
group by txn_type
order by transaction_count desc;
```

```
select channel, count(*) AS transaction_count
from transactions
group by channel
order by transaction_count desc;
```

--customers have at least one account

```
select count(distinct customer_id) as customer_with_accounts
from accounts;
```

--customers have loans

```
select count(distinct customer_id) as customer_with_loans
from loans;
```

--customers have cards

```
select count(distinct customer_id) as customer_with_cards
from cards;
```

--accounts have transactions

```
select count(distinct account_id) as customer_with_accounts
```

```
from accounts;
```

```
--loans have payment records
```

```
select count(distinct loan_id) as customer_with_loan  
from loans;
```

A.2 Data Quality and Validation Checks

```
select count(*) as missing_income  
from customers  
where annual_income is null;
```

```
select customer_id, count(*) as duplicate_count  
from customers  
group by customer_id  
having count(*) > 1;
```

```
select count(*) as orphan_accounts  
from accounts a  
left join customers c  
ON a.customer_id = c.customer_id  
where c.customer_id is null;
```

```
select count(*) as invalid_balance  
from accounts  
where balance < 0;
```

A.3 Customer Analysis

```
--How many customers does the bank have?
```

```
select count(*) as total_customers  
from customers;
```

```
--Which states have the most customers?
```

```
select * from customers;  
select state, count(distinct customer_id) as total_customers
```

```

from customers

group by state

order by total_customers desc;

--How are customers distributed by income?

select

case

    when annual_income < 500000 then 'below 5lakh'

    when annual_income < 1000000 then ' 5 - 10lakh'

    when annual_income < 2000000 then '10 - 20 5lakh'

    when annual_income < 5000000 then '20 - 50lakh'

    else 'above 50 lakh'

end as income_distribution ,

count(customer_id) as customer_count

from customers

group by case

    when annual_income < 500000 then 'below 5lakh'

    when annual_income < 1000000 then ' 5 - 10lakh'

    when annual_income < 2000000 then '10 - 20 5lakh'

    when annual_income < 5000000 then '20 - 50lakh'

    else 'above 50 lakh'

end

order by income_distribution desc;

--What is the average credit score?

select * from customers

select avg(credit_score) as average_credit_score

from customers;

--How are customers joining over time?

select year(join_date) as join_year,

count(customer_id) as customer_count

from customers

group by year(join_date )

```

```

order by join_year;

---Which customers have the highest annual income?

select top 10 customer_id,name,annual_income
from customers
order by annual_income desc;

---Are high-income customers concentrated in particular locations?

with high_income_customers as
(
select customer_id,name,state,annual_income
from customers
where annual_income >
    (select avg(cast(annual_income as decimal(18,2))) from customers)
)
select state,count(customer_id)as high_income_customers,
    avg(annual_income) as avg_high_income
from high_income_customers
group by state
order by high_income_customers desc;

---Highest-credit-score customers

select top 20 customer_id,name,credit_score
from customers
order by credit_score desc;

--Which occupations have higher-income customers?

select occupation,
count(customer_id) as customer_count,
avg(cast(annual_income as decimal(18,2))) as avg_high_income
from customers
group by occupation
order by avg_high_income desc;

```

A.4 Account Analysis

```

--Which account types are most popular?

```

```

select * from accounts;

select account_type,
       count( customer_id) as customer_count
from accounts
group by account_type
order by customer_count desc;

--Which account types hold the highest balances?
select account_type,sum(balance) as total_balance
from accounts
group by account_type
order by total_balance desc;

--How many accounts are Active, Dormant and Closed?
select status,count(distinct account_id) as accounts_count
from accounts
group by status
order by accounts_count desc;

---What is the total account balance?
select sum(balance) as total_account_balance
from accounts

--Which customers maintain multiple accounts?
select c.customer_id,c.name, count(a.account_id) as number_of_accounts
from customers c
inner join accounts a
on c.customer_id = a.customer_id
group by c.customer_id,c.name
having count(a.account_id) >=2
order by count(a.account_id) desc;

--Which branches manage the highest account balances?
select * from accounts;
select * from branches;

```

```

select b.branch_name, sum(a.balance)as total_balance
from accounts a
inner join branches b
on a.branch_id = b.branch_id
group by branch_name
order by total_balance desc;

```

---How has account opening changed over time?

```

select year(open_date) as opened_year,
count(account_id ) as number_of_accounts
from accounts
group by year(open_date)
order by opened_year;

```

A.5 Transaction Analysis

--What is the total transaction value?

```

select* from transactions;

select sum(amount) as total_transaction_value
from transactions;

```

--What is the transaction volume?

```

select count(transaction_id)as transaction_volume
from transactions;

```

--Which transaction types are most common?

```

select txn_type ,count(account_id) as number_of_accounts
from transactions
group by txn_type
order by number_of_accounts desc;

```

--Which channels are most frequently used?

```

select channel,count(account_id) as number_of_accounts
from transactions
group by channel
order by number_of_accounts desc;

```

--How do deposits compare with withdrawals?

```

select txn_type, sum(amount) as total_amount
from transactions
where txn_type in ('deposit','withdrawal')
group by txn_type
order by total_amount desc;

--How frequently are deposits and withdrawals happening?

```

```

select txn_type, sum(amount) as total_amount,
count(transaction_id) as transaction_volume
from transactions
where txn_type in ('deposit','withdrawal')
group by txn_type
order by total_amount desc;

```

--Which months have the highest transaction activity?

```

select year(txn_date) as transaction_year,
       MONTH(txn_date) as transaction_month,
       count(transaction_id) as transaction_volume,
       sum(amount) as transaction_value
from transactions
group by year(txn_date), MONTH(txn_date)
order by transaction_volume desc;

```

--Which customers have the highest transaction activity?

```

select top 15 c.customer_id, c.name, sum(amount) as transaction_value,
count(t.transaction_id) as transaction_volume
from accounts a
inner join transactions t
on a.account_id = t.account_id
inner join customers c
on a.customer_id = c.customer_id
group by c.customer_id, c.name
order by transaction_value desc

```

--Which branches generate the highest transaction value?


```

select * from branches;

select top 10 b.branch_name,b.state,sum(t.amount) as transaction_value
from accounts a
inner join branches b
on a.branch_id = b.branch_id
inner join transactions t
on a.account_id=t.account_id
group by b.branch_name,b.state
order by transaction_value desc;

```

A.6 Loan Analysis

--What is the total loan amount?

```

select * from loans

select sum(loan_amount) as total_loan_amount
from loans;

```

--Which loan types are most common?

```

select loan_type,count(loan_id) as number_of_loans
from loans

group by loan_type
order by number_of_loans desc;

```

--Which loan types have the highest value?

```

select loan_type,sum(loan_amount) as total_loan_amount
from loans

group by loan_type
order by total_loan_amount desc;

```

--How many loans are Active, Closed, Defaulted and Written Off?

```

select status,count(loan_id) as number_of_loans
from loans

group by status
order by number_of_loans desc

```

---What is the average loan amount?

```

select avg(loan_amount) as average_loan_amount

```

```
from loans
```

```
--Which states have the highest loan exposure?
```

```
select c.state,
```

```
sum(loan_amount)as total_loan_amount,
```

```
count(loan_id) as no_of_loans
```

```
from loans l
```

```
inner join customers c
```

```
on l.customer_id = c.customer_id
```

```
group by c.state
```

```
order by total_loan_amount desc;
```

```
--How are loan payments progressing?
```

```
select* from loan_payments
```

```
select year(payment_date) as payment_year,
```

```
count(payment_id) as no_of_payments,
```

```
sum(amount_paid) as total_amount_paid,
```

```
sum(principal_component) as total_principal_paid,
```

```
sum(interest_component) as total_interest_paid,
```

```
sum
```

```
(case
```

```
when late_payment_flag = 1 then 1
```

```
else 0
```

```
end) as late_payments
```

```
from loan_payments
```

```
group by year(payment_date)
```

```
order by payment_year;
```

```
--How has lending changed over time?
```

```
select year(start_date) as loan_year,
```

```
count(loan_id) as number_of_loans,
```

```
sum(loan_amount) as total_loan_amount,
```

```
Avg(loan_amount) as average_loan_amount
```

```
from loans
```

```
group by YEAR(start_date)
```

```
order by loan_year;
```

A.7 Risk Analysis

--What percentage of loans are defaulted?

```
select count(loan_id) as num_of_loans,  
       count(case when status = 'defaulted' then 1 end) as defaulted_loans,  
       round(count(case when status = 'defaulted' then 1 end) * 100.00 / count(loan_id), 2) as defaulted_loan_percentage  
from loans;
```

--Which loan types have the highest default rate?

```
select * from loans  
  
select loan_type, count(loan_id) as num_of_loans,  
       count(case when status = 'defaulted' then 1 end) as defaulted_loans,  
       round(count(case when status = 'defaulted' then 1 end) * 100.00 / count(loan_id), 2) as defaulted_loan_percentage  
from loans  
  
group by loan_type  
  
order by defaulted_loan_percentage desc;
```

--Which states have higher default rates?

```
select c.state, count(l.loan_id) as num_of_loans,  
       count(case when l.status = 'defaulted' then 1 end) as defaulted_loans,  
       round(count(case when l.status = 'defaulted' then 1 end) * 100.00 / count(l.loan_id), 2) as defaulted_loan_percentage  
from loans l  
  
inner join customers c  
on l.customer_id = c.customer_id  
  
group by c.state  
  
order by defaulted_loan_percentage desc;
```

--How much money is associated with defaulted loans?

```
select count(loan_id) as num_of_loans,  
       sum(loan_amount) as total_defaulted_loans,  
       avg(loan_amount) as average_defaulted_loans
```

```

from loans

where status = 'defaulted'

--How much has been written off?

select count(status) as written_off_accounts

from loans

where status = 'written off'

--Which customers have poor credit scores?

select * from customers

select customer_id,name,credit_score

from customers

where credit_score < 599

order by credit_score;

--Are low-credit-score customers associated with higher loan risk?

select case

    when c.credit_score < 599 then 'poor'

    when c.credit_score <700 then 'good'

    when c.credit_score < 800 then 'fair'

    else 'excellent'

end as credit_score_segmentation,

count(case when l.status = 'defaulted' then 1 end) as defaulted_loans,

round(count(case when l.status = 'defaulted' then 1 end) * 100.00 / count(l.loan_id),2) as defaulted_rate

from loans l

inner join customers c

on l.customer_id = c.customer_id

group by case

    when c.credit_score < 599 then 'poor'

    when c.credit_score <700 then 'good'

    when c.credit_score < 800 then 'fair'

    else 'excellent'

end

order by defaulted_rate desc

```

--Are there unusual/fraudulent card transactions?

```
select * from card_transactions

select

    count(card_txn_id) as total_card_transactions,

    count(case when is_fraud = 1 then 1 end) as fraudulent_transactions,

    round(count(case when is_fraud = 1 then 1 end) * 100.0 / count(card_txn_id),2) as fraud_percentage

from card_transactions;
```

--How are late loan payments distributed?

```
select

    year(payment_date) as payment_year,

    count(payment_id) as total_payments,

    count(case when late_payment_flag = 1 then 1 end) as late_payments,

    round ( count(case when late_payment_flag = 1 then 1 end) * 100.0 / count(payment_id), 2) as late_payment_percentage

from loan_payments

group by year(payment_date)

order by payment_year;
```

A.8 Branch Analysis

--Which branches have the most customers?

```
select * from branches;

select b.branch_name,

    count(distinct c.customer_id) as number_of_customers

from accounts a

inner join branches b

on a.branch_id = b.branch_id

inner join customers c

on a.customer_id = c.customer_id

group by branch_name

order by number_of_customers desc;
```

--Which branches manage the highest account balances?

```
select * from accounts

select top 15 b.branch_name,sum(a.balance) as total_balance
```

```

from accounts a
inner join branches b
on a.branch_id = b.branch_id
group by b.branch_name
order by total_balance desc;

--Which branches generate the most transactions?

select * from transactions
select b.branch_name,count(t.transaction_id) as no_of_transactions
from accounts a
inner join transactions t
on a.account_id = t.account_id
inner join branches b
on a.branch_id = b.branch_id
group by b.branch_name
order by no_of_transactions desc;

--Which branches have the highest loan portfolio?

select * from loans
select b.branch_name,sum(l.loan_amount) as total_loan_amount
from branches b
inner join loans l
on b.branch_id = l.branch_id
group by b.branch_name
order by total_loan_amount desc;

--Which branches have the highest number of accounts?

select b.branch_name,count(a.account_id) as num_of_account
from accounts a
inner join branches b
on a.branch_id = b.branch_id
group by b.branch_name
order by num_of_account desc;

--How do branches rank against each other?

```

```

select b.branch_name,count(distinct c.customer_id) as no_of_customer,
       rank()over ( order by count(distinct c.customer_id) desc) as branch_rank
from accounts a
inner join customers c
on a.customer_id = c.customer_id
inner join branches b
on a.branch_id = b.branch_id
group by b.branch_name

```

--Which states have the strongest branch performance?

```

select * from accounts
select c.state,
       count(distinct c.customer_id) as num_of_customers,
       count(distinct a.account_id) as num_of_accounts,
       sum(balance) as total_balance
from accounts a
inner join branches b
on a.branch_id = b.branch_id
inner join customers c
on a.customer_id = c.customer_id
group by c.state,b.branch_name
order by total_balance desc;

```

A.9 Card Analysis

--How many cards have been issued?

```

select * from cards
select count(distinct card_id) as num_of_cards
from cards;

```

--Which card types are most popular?

```

select card_type,count(distinct card_id) as no_of_cards
from cards
group by card_type
order by no_of_cards desc;

```

--How many cards are active/inactive?

```
select status ,  
       count(distinct card_id) as num_of_cards  
from cards  
group by status  
order by num_of_cards desc;
```

--What is the total card transaction value?

```
select * from card_transactions  
select sum(amount) as total_card_transaction_value  
from card_transactions;
```

--Which channels are used for card transactions?

```
select * from transactions  
select * from card_transactions  
select merchant_category,  
       count(distinct card_txn_id) as number_of_card_transactions  
from card_transactions  
group by merchant_category  
order by number_of_card_transactions desc;
```

--What is the monthly card spending trend?

```
select year(txn_date) as transaction_year,  
       month(txn_date) as transaction_month,  
       sum(amount) as total_amount  
from card_transactions  
group by year(txn_date),month(txn_date)  
order by total_amount desc;
```

--Which customers have the highest card activity?

```
select top 15 c.customer_id,c.name,  
       count(ct.card_txn_id) as card_transaction_count  
from customers c  
inner join cards ca  
on c.customer_id = ca.customer_id
```



```

inner join card_transactions ct
    on ca.card_id = ct.card_id
group by c.customer_id,c.name
order by card_transaction_count desc;

--Are there suspicious/fraudulent card transactions?
select count(card_txn_id) as total_card_transactions,
    count(case when is_fraud = 1 then 1 end) as fraudulent_transactions,
    round( count(case when is_fraud = 1 then 1 end) * 100.0 / count(card_txn_id),2 ) as fraud_percentage
from card_transactions;

```

A.10 Support Analysis

```

--How many support tickets were raised?
select * from support_tickets

select count(distinct ticket_id) as number_of_support_tickets
from support_tickets

--Which issue types are most common?
select issue_type,
    count(distinct ticket_id)as number_of_support_tickets
from support_tickets
group by issue_type
order by number_of_support_tickets desc;

-How many tickets are Open, Pending and Resolved?
select status,count(distinct ticket_id) as num_of_support_tickets
from support_tickets
group by status
order by num_of_support_tickets desc;

--Which branches/customers generate more tickets?
select b.branch_name,count(distinct s.customer_id) as no_of_customers,
    count(distinct ticket_id) as num_of_support_tickets
from accounts a
inner join support_tickets s
on a.customer_id = s.customer_id

```

```

inner join branches b
on b.branch_id = a.branch_id
group by b.branch_name
order by num_of_support_tickets desc;

select top 15 s.customer_id,c.name,
    count(distinct s.ticket_id) as num_of_support_tickets
from support_tickets s
inner join customers c
on s.customer_id = c.customer_id
group by s.customer_id, c.name
order by num_of_support_tickets desc;

--How long does ticket resolution take?
select ticket_id,
    datediff(day, date_opened,date_resolved) as resolution_days
from support_tickets
where date_resolved is not null
order by resolution_days desc;

--average resolution takes
select avg (cast (datediff(day,date_opened,date_resolved) as decimal(18,2))) as avg_resolution_days
from support_tickets
where date_resolved is not null;

--compare resolution time by issue type
select issue_type,
    count(distinct ticket_id) as num_of_tickets,
    avg(cast(datediff(day,date_opened,date_resolved) as decimal(10,2))) as avg_resolution_days
from support_tickets
where date_resolved is not null
group by issue_type
order by avg_resolution_days desc;

--What is the customer satisfaction level?

```

```

select count(satisfaction_score) AS rated_tickets,
       avg(satisfaction_score) as avg_satisfaction_level
from support_tickets
where satisfaction_score is not null

--Has support volume increased over time?

select year(date_opened) as opened_year, month(date_opened) as opened_month,
       count(distinct ticket_id) as no_of_tickets
from support_tickets
group by year(date_opened), month(date_opened)
order by opened_year, opened_month;

--Which issues have the lowest satisfaction?

select issue_type,
       avg(satisfaction_score) as avg_satisfaction_score
from support_tickets
where satisfaction_score is not null
group by issue_type
order by avg_satisfaction_score;

```

A.11 Result Validation

```

--customer validation

-- total customers

select count(distinct customer_id)
from customers;

-- credit score range

select
    min(credit_score) as min_credit_score,
    max(credit_score) as max_credit_score,
    avg(credit_score) as avg_credit_score
from customers;

--account validation

-- total accounts

```

```

select count(distinct account_id)
from accounts;

-- account status totals

select
    status,
    count(distinct account_id) as number_of_accounts
from accounts
group by status;

--loan validation

-- total loans

select count(distinct loan_id)
from loans;

-- loan status totals

select
    status,
    count(distinct loan_id) as number_of_loans
from loans
group by status;

--transaction validation

-- transaction count and total value

select
    count(distinct transaction_id) as transaction_volume,
    sum(amount) as transaction_value
from transactions;

--support validation

-- total tickets

select count(distinct ticket_id)
from support_tickets;

-- ticket status distribution

```

```

select
    status,
    count(distinct ticket_id) as ticket_count
from support_tickets
group by status;

--validate module 1-- 1. credit score range
select
    min(credit_score) as min_credit_score,
    max(credit_score) as max_credit_score,
    avg(credit_score) as avg_credit_score
from customers;

-- 2. income range
select
    min(annual_income) as min_income,
    max(annual_income) as max_income,
    avg(annual_income) as avg_income
from customers;

-- 3. total customers by joining year
select
    year(join_date) as join_year,
    count(distinct customer_id) as customer_count
from customers
group by year(join_date)
order by join_year;

select
    min(txn_date) as min_date,
    max(txn_date) as max_date,
    sum(amount) as total_transaction_value
from transactions
where year(txn_date) = 2026;

```

APPENDIX B – DAX MEASURES AND CALCULATIONS

B.1 Customer Analysis Measures

Total Customers =

DISTINCTCOUNT(customers[customer_id])

Average Annual Income =

AVERAGE(customers[annual_income])

Average Credit Score =

AVERAGE(customers[credit_score])

B.2 Account Analysis Measures

Total Accounts =

DISTINCTCOUNT(accounts[account_id])

Total Account Balance =

SUM(accounts[balance])

Average Account Balance =

AVERAGE(accounts[balance])

Active Accounts =

CALCULATE(
 DISTINCTCOUNT(accounts[account_id]),
 accounts[status] = "Active"
)

B.3 Transaction Analysis Measures

Total Transactions =

COUNT(transactions[transaction_id])

Total Transaction Value =

SUM(transactions[amount])

Average Transaction Amount =

AVERAGE(transactions[amount])

B.4 Loan and Risk Analysis Measures

Total Loans =

DISTINCTCOUNT(loans[loan_id])

Total Loan Amount =

SUM(loans[loan_amount])

Average Loan Amount =

AVERAGE(loans[loan_amount])

Default Rate =

DIVIDE(

 CALCULATE(

 DISTINCTCOUNT(loans[loan_id]),

 loans[status] = "Default"

),

 [Total Loans],

 0

)

B.5 Branch Analysis Measures

Total Branches =

DISTINCTCOUNT(branches[branch_id])

Total Branch Customers =

DISTINCTCOUNT(customers[customer_id])

Total Branch Balance =

SUM(accounts[balance])

Branch Transactions Display =

FORMAT(

 DIVIDE([Branch Transactions], 1000),

 "0.00"

) & "K"

B.6 Card Analysis Measures

Total Cards =

DISTINCTCOUNT(cards[card_id])

Active Cards =

CALCULATE(

```

    DISTINCTCOUNT(cards[card_id]),
    cards[status] = "Active"
)
Total Card Transaction Value =
SUM(card_transactions[amount])
Fraud Transactions =
SUM(card_transactions[is_fraud])
Fraud % =
DIVIDE(
    [Fraud Transactions],
    COUNT(card_transactions[card_txn_id]),
    0
)
Card Transaction Count =
VAR CustomerID =
    SELECTEDVALUE(customers[customer_id])
RETURN
    CALCULATE(
        COUNT(card_transactions[card_txn_id]),
        REMOVEFILTERS(accounts),
        FILTER(
            ALL(cards),
            cards[customer_id] = CustomerID
        )
    )
)

```

B.7 Support Analysis Measures

```

Total Support Tickets =
COUNT(support_tickets[ticket_id])
Average Resolution Days =

```



```

AVERAGEX(
  FILTER(
    support_tickets,
    NOT ISBLANK(support_tickets[date_resolved])
  ),
  DATEDIFF(
    support_tickets[date_opened],
    support_tickets[date_resolved],
    DAY
  )
)
Average Satisfaction Score =
AVERAGE(support_tickets[satisfaction_score])

```

B.8 Calculated Columns

Credit Segment

Credit Segment =

```

SWITCH(
  TRUE(),
  [credit_score] < 580, "Poor",
  [credit_score] < 700, "Good",
  [credit_score] < 800, "Fair",
  "Excellent"
)

```

Fraud Status

Fraud Status =

```

IF(
  card_transactions[is_fraud] = 1,
  "Fraud",
  "Legitimate"
)

```

Transaction Year

```
Transaction Year =  
YEAR(card_transactions[txn_date])  
Support Year  
Support Year =  
YEAR(support_tickets[date_opened])
```

B.9 Display Formatting Measures

```
Avg Balance Display =  
FORMAT(  
    DIVIDE([Average Account Balance], 1000),  
    "0.00"  
) & "K"
```